EXPERIMENT NO. 10

Character, Digit, or Face Classification using Convolutional Neural Networks (CNNs)

Aim

To design, train, and evaluate a Convolutional Neural Network (CNN) for character/digit classification using a standard image dataset and analyze its performance using appropriate evaluation metrics.

Objective

In this experiment, we will:

Load the MNIST handwritten digit dataset.

Preprocess and normalize images.

Design a CNN model.

Train the CNN.

Plot training and validation accuracy/loss.

Test the trained model.

Calculate Accuracy, Precision, Recall and F1-Score.

Generate a Confusion Matrix.

Predict an unseen handwritten digit using a manually specified image path.

The experiment sheet lists MNIST as a standard dataset for handwritten digit recognition.

Theory

What is CNN?

A Convolutional Neural Network (CNN) is a deep learning model mainly used for image-related tasks.

A CNN automatically learns image features such as:

Edges Corners Shapes Textures Complex patterns

The experiment sheet specifically describes CNNs as automatically learning hierarchical image features instead of requiring handcrafted feature extraction.

Main CNN components

1. Convolution Layer

   Extracts important features from the image using filters.

2. Pooling Layer

   Reduces the spatial size of feature maps while retaining important information.

3. Fully Connected Layer

   Uses the extracted features to perform classification.

4. Output Layer

   Produces the final class probabilities.

5. Dataset MNIST

   MNIST contains handwritten digits from:

   0 1 2 3 4 5 6 7 8 9

Each image is:

28 × 28 pixels

There are 10 classes, corresponding to the ten digits.

6. Complete Google Colab Code Cell 1 — Import Libraries

Gemini

```
# ============================================================
# EXPERIMENT 10
# CHARACTER / DIGIT CLASSIFICATION USING CNN
# Dataset: MNIST
# ============================================================

import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
import cv2
import os

from tensorflow.keras import Sequential
from tensorflow.keras.layers import (
    Conv2D,
    MaxPooling2D,
    Flatten,
    Dense,
    Dropout
)

from tensorflow.keras.datasets import mnist

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

print("All libraries imported successfully.")
```

```
All libraries imported successfully.
```

Cell 2 — Load Dataset

```
# ============================================================
# LOAD MNIST DATASET
# ============================================================

print("Loading MNIST dataset...")

(x_train, y_train), (x_test, y_test) = mnist.load_data()

print("\nDataset loaded successfully!")

print("Training images :", x_train.shape)
print("Training labels :", y_train.shape)
print("Testing images  :", x_test.shape)
print("Testing labels  :", y_test.shape)
```

```
Loading MNIST dataset...
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step

Dataset loaded successfully!
Training images : (60000, 28, 28)
Training labels : (60000,)
Testing images  : (10000, 28, 28)
Testing labels  : (10000,)
```

Expected output Training images : (60000, 28, 28) Training labels : (60000,) Testing images : (10000, 28, 28) Testing labels : (10000,) Cell 3 — Display Sample Images

```
# ============================================================
# DISPLAY SAMPLE IMAGES
# ============================================================

plt.figure(figsize=(10, 5))

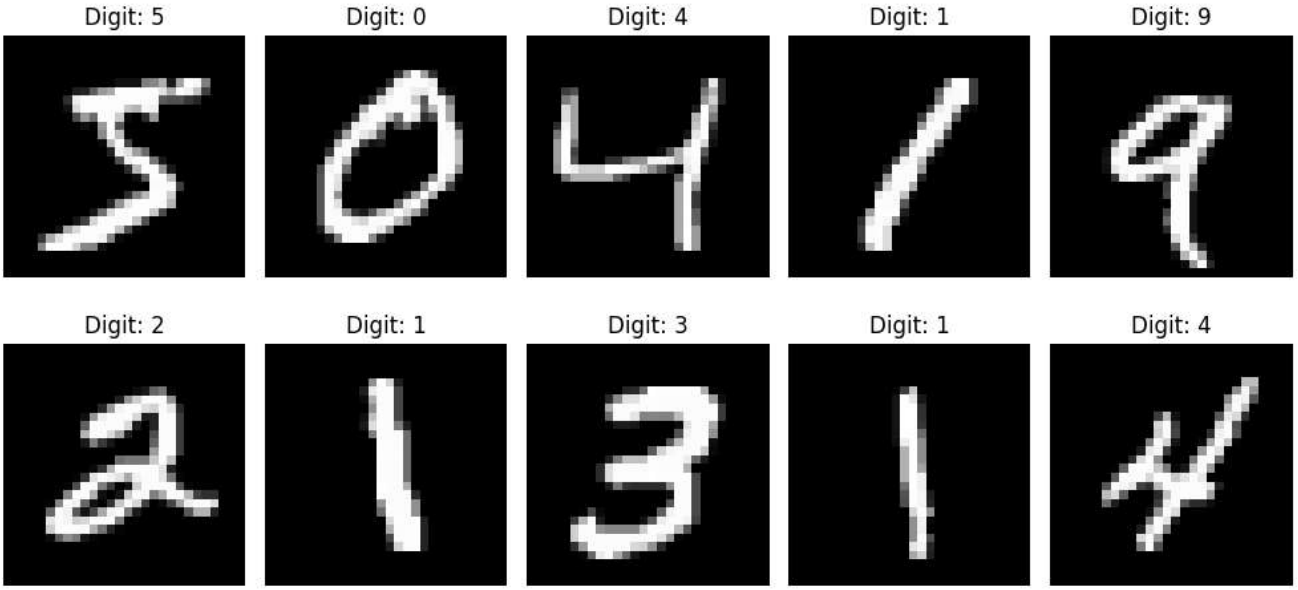
for i in range(10):

    plt.subplot(2, 5, i + 1)

    plt.imshow(
        x_train[i],
        cmap="gray"
    )

    plt.title("Digit: " + str(y_train[i]))
    plt.axis("off")

plt.tight_layout()
plt.show()
```



Cell 4 — Image Preprocessing

The experiment requires preprocessing including normalization and splitting into training, validation and testing data.

```
# ============================================================
# IMAGE PREPROCESSING
# ============================================================

# Convert pixel values from 0-255 to 0-1
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# Add channel dimension
x_train = np.expand_dims(x_train, axis=-1)
x_test = np.expand_dims(x_test, axis=-1)

print("After preprocessing:")
print("Training shape:", x_train.shape)
print("Testing shape :", x_test.shape)
```

```
After preprocessing:
Training shape: (60000, 28, 28, 1)
Testing shape : (10000, 28, 28, 1)
```

Cell 5 — Create CNN Model

The required CNN structure includes convolution, pooling, activation, fully connected and output layers.

```
# ============================================================
# CREATE CNN MODEL
# ============================================================

model = Sequential([

    # First Convolution Layer
    Conv2D(
        32,
        kernel_size=(3, 3),
        activation="relu",
        input_shape=(28, 28, 1)
    ),

    # First Pooling Layer
    MaxPooling2D(
        pool_size=(2, 2)
    ),

    # Second Convolution Layer
    Conv2D(
        64,
        kernel_size=(3, 3),
        activation="relu"
    ),

    # Second Pooling Layer
    MaxPooling2D(
        pool_size=(2, 2)
    ),

    # Flatten
    Flatten(),

    # Fully Connected Layer
    Dense(
        128,
        activation="relu"
    ),

    # Dropout
    Dropout(0.5),

    # Output Layer
    Dense(
        10,
        activation="softmax"
    )
])

print("CNN model created successfully!")
```

```
CNN model created successfully!
/usr/local/lib/python3.13/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shap
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Cell 6 — Display Model

```
# ============================================================
# MODEL SUMMARY
# ============================================================

model.summary()
```

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290

```
Total params: 225,034 (879.04 KB)
Trainable params: 225,034 (879.04 KB)
```

Cell 7 — Compile Model

```
# ============================================================
# COMPILE CNN
# ============================================================

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

print("Model compiled successfully!")
```

```
Model compiled successfully!
```

The experiment requires configuration of optimizer, loss function, learning rate, batch size and epochs.

Cell 8 — Train Model

```
# ============================================================
# TRAIN CNN MODEL
# ============================================================

history = model.fit(
    x_train,
    y_train,

    validation_split=0.1,

    epochs=5,

    batch_size=128,

    verbose=1
)

print("\nTraining completed successfully!")
```

```
Epoch 1/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 54s 121ms/step - accuracy: 0.8991 - loss: 0.3321 - val_accuracy: 0.9815 - val_loss: 0.0683
Epoch 2/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 72s 98ms/step - accuracy: 0.9668 - loss: 0.1118 - val_accuracy: 0.9873 - val_loss: 0.0450
Epoch 3/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 50s 117ms/step - accuracy: 0.9756 - loss: 0.0812 - val_accuracy: 0.9888 - val_loss: 0.0407
Epoch 4/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 46s 110ms/step - accuracy: 0.9797 - loss: 0.0677 - val_accuracy: 0.9908 - val_loss: 0.0320
Epoch 5/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 86s 119ms/step - accuracy: 0.9830 - loss: 0.0571 - val_accuracy: 0.9903 - val_loss: 0.0345

Training completed successfully!
```

The experiment requires monitoring training and validation accuracy/loss during training.

Cell 9 — Accuracy Graph

```
# ============================================================
# TRAINING AND VALIDATION ACCURACY
# ============================================================

plt.figure(figsize=(8, 5))

plt.plot(
    history.history["accuracy"],
    label="Training Accuracy"
)

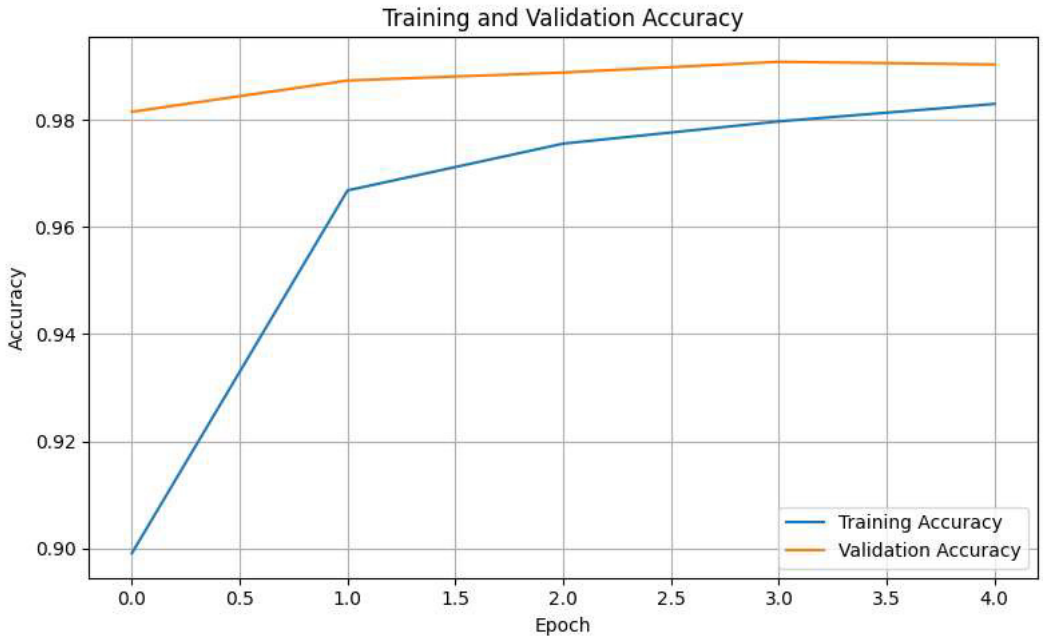
plt.plot(
    history.history["val_accuracy"],
    label="Validation Accuracy"
)

plt.xlabel("Epoch")
plt.ylabel("Accuracy")

plt.title(
    "Training and Validation Accuracy"
)

plt.legend()
plt.grid()

plt.tight_layout()
plt.show()
```



Cell 10 — Loss Graph

```
# ============================================================
# TRAINING AND VALIDATION LOSS
# ============================================================

plt.figure(figsize=(8, 5))

plt.plot(
    history.history["loss"],
    label="Training Loss"
)

plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
```

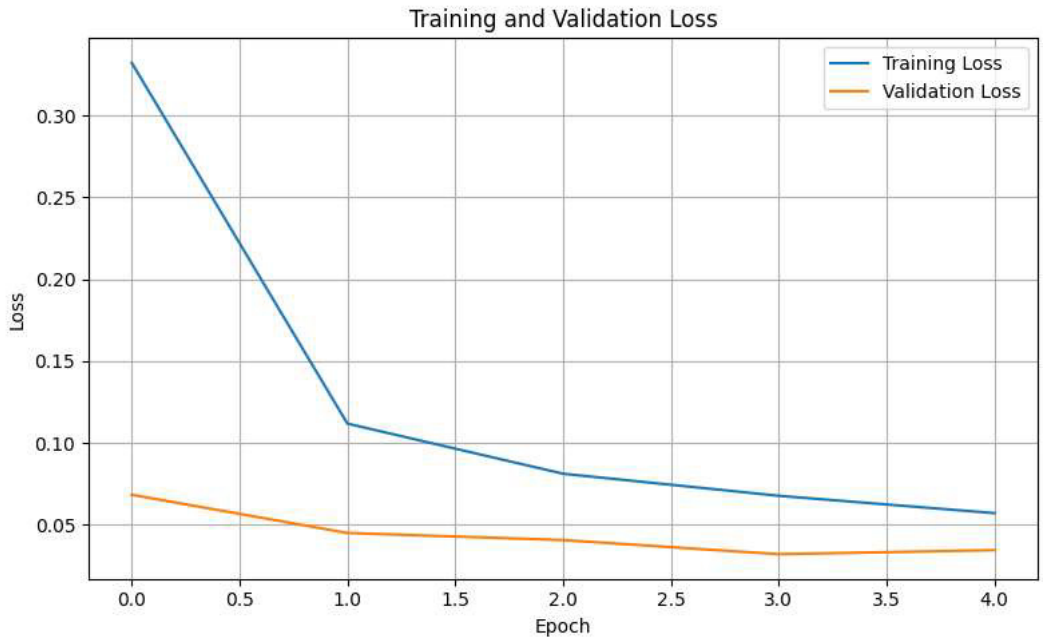
```
)

plt.xlabel("Epoch")
plt.ylabel("Loss")

plt.title(
    "Training and Validation Loss"
)

plt.legend()
plt.grid()

plt.tight_layout()
plt.show()
```



Cell 11 — Evaluate Model

```
# ============================================================
# MODEL EVALUATION
# ============================================================

test_loss, test_accuracy = model.evaluate(
    x_test,
    y_test,
    verbose=0
)

print("\n================================")
print("       MODEL EVALUATION")
print("================================")

print("Test Loss     :", test_loss)
print("Test Accuracy :", test_accuracy)
```

```
================================
       MODEL EVALUATION
================================
Test Loss     : 0.030814923346042633
Test Accuracy : 0.9900000095367432
```

Cell 12 — Predictions

```
# ============================================================
# MAKE PREDICTIONS
# ============================================================

y_probability = model.predict(
```

```
    x_test,
    verbose=0
)

y_pred = np.argmax(
    y_probability,
    axis=1
)

print("Predictions generated successfully!")
```

```
Predictions generated successfully!
```

Cell 13 — Accuracy, Precision, Recall, F1

The experiment specifically requires these performance metrics.

```
# ============================================================
# PERFORMANCE METRICS
# ============================================================

accuracy = accuracy_score(
    y_test,
    y_pred
)

precision = precision_score(
    y_test,
    y_pred,
    average="weighted"
)

recall = recall_score(
    y_test,
    y_pred,
    average="weighted"
)

f1 = f1_score(
    y_test,
    y_pred,
    average="weighted"
)

print("\n================================")
print("      PERFORMANCE METRICS")
print("================================")

print(
    "Accuracy  :",
    round(accuracy, 4)
)

print(
    "Precision :",
    round(precision, 4)
)

print(
    "Recall    :",
    round(recall, 4)
)

print(
    "F1-Score  :",
    round(f1, 4)
)
```

```

================================
      PERFORMANCE METRICS
================================
Accuracy  : 0.99
Precision : 0.99
Recall    : 0.99
F1-Score  : 0.99
```

Cell 14 — Confusion Matrix

```
# ============================================================
# CONFUSION MATRIX
# ============================================================

cm = confusion_matrix(
    y_test,
    y_pred
)

plt.figure(figsize=(8, 7))

plt.imshow(
    cm,
    cmap="Blues"
)

plt.title(
    "Confusion Matrix"
)

plt.xlabel(
    "Predicted Label"
)

plt.ylabel(
    "Actual Label"
)

plt.colorbar()

plt.xticks(
    range(10)
)

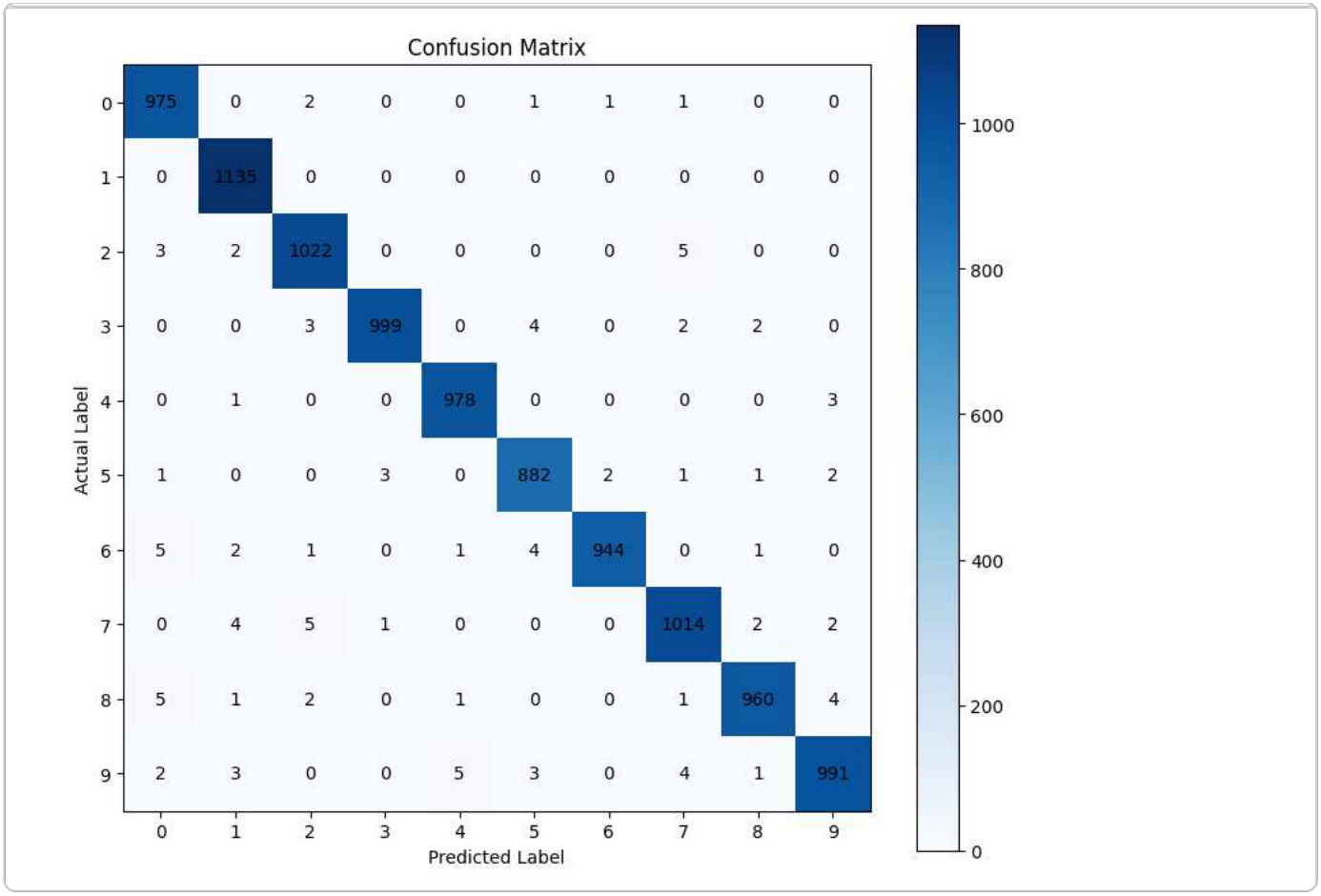
plt.yticks(
    range(10)
)

for i in range(10):

    for j in range(10):

        plt.text(
            j,
            i,
            cm[i, j],
            ha="center",
            va="center"
        )

plt.tight_layout()
plt.show()
```



A confusion matrix helps identify correctly classified and misclassified classes. The experiment sheet explicitly asks for its use in evaluating classification performance.

Cell 15 — Classification Report

```
# ============================================================
# CLASSIFICATION REPORT
# ============================================================

print("\n================================")
print("     CLASSIFICATION REPORT")
print("================================")

print(
    classification_report(
        y_test,
        y_pred,
        digits=4
    )
)
```

```
================================
     CLASSIFICATION REPORT
================================
              precision    recall  f1-score   support

           0     0.9839    0.9949    0.9893       980
           1     0.9887    1.0000    0.9943      1135
           2     0.9874    0.9903    0.9889      1032
           3     0.9960    0.9891    0.9925      1010
           4     0.9929    0.9959    0.9944       982
           5     0.9866    0.9888    0.9877       892
           6     0.9968    0.9854    0.9911       958
           7     0.9864    0.9864    0.9864      1028
           8     0.9928    0.9856    0.9892       974
           9     0.9890    0.9822    0.9856      1009

    accuracy                         0.9900     10000
   macro avg     0.9900    0.9899    0.9899     10000
```

```
weighted avg     0.9900    0.9900    0.9900     10000
```

Cell 16 — Show Predictions

The experiment asks for predictions on unseen images and comparison of predicted and actual labels.

```
# ============================================================
# SHOW PREDICTIONS
# ============================================================

plt.figure(figsize=(12, 8))

for i in range(12):

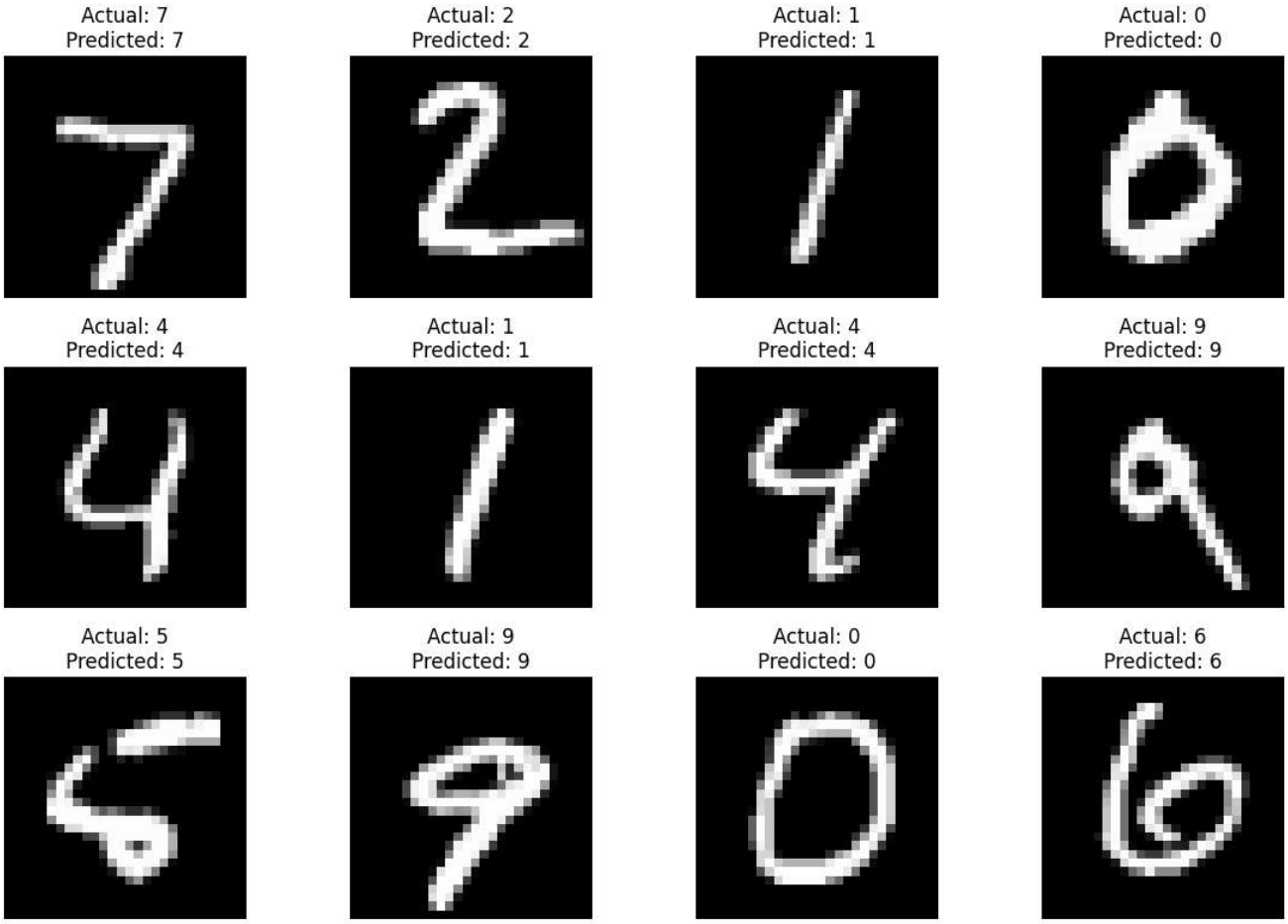
    plt.subplot(3, 4, i + 1)

    plt.imshow(
        x_test[i].squeeze(),
        cmap="gray"
    )

    plt.title(
        "Actual: " + str(y_test[i]) +
        "\nPredicted: " + str(y_pred[i])
    )

    plt.axis("off")

plt.tight_layout()
plt.show()
```



Cell 17 — MANUAL IMAGE PATH

```
# ============================================================
# MANUAL IMAGE PATH
# ============================================================

image_path = "/content/Screenshot 2026-09-30 004958.png"

print("Image Path:")
print(image_path)
```

```
Image Path:
/content/Screenshot 2026-09-30 004958.png
```

Cell 18 — Predict Your Own Image

```
# ============================================================
# CUSTOM IMAGE PREDICTION
# ============================================================

if not os.path.exists(image_path):

    print("\nERROR!")
    print("Image not found.")
    print("Please check the image path.")

else:

    # Read image as grayscale
    custom_image = cv2.imread(
        image_path,
        cv2.IMREAD_GRAYSCALE
    )

    if custom_image is None:

        print("ERROR: Could not read image.")

    else:

        # Resize image to MNIST size
        custom_image = cv2.resize(
            custom_image,
            (28, 28)
        )

        # Convert to float
        custom_image = custom_image.astype(
            "float32"
        ) / 255.0

        # Invert if required
        # MNIST uses white digit on black background
        if np.mean(custom_image) > 0.5:

            custom_image = 1.0 - custom_image

        # Reshape
        custom_input = custom_image.reshape(
            1,
            28,
            28,
            1
        )

        # Prediction
        prediction = model.predict(
            custom_input,
            verbose=0
        )

        predicted_digit = np.argmax(
            prediction[0]
        )
```

```
        confidence = np.max(
            prediction[0]
        )

        # Display
        plt.figure(figsize=(5, 5))

        plt.imshow(
            custom_image,
            cmap="gray"
        )

        plt.title(
            "Predicted Digit: "
            + str(predicted_digit)
            + "\nConfidence: "
            + f"{confidence * 100:.2f}%"
        )

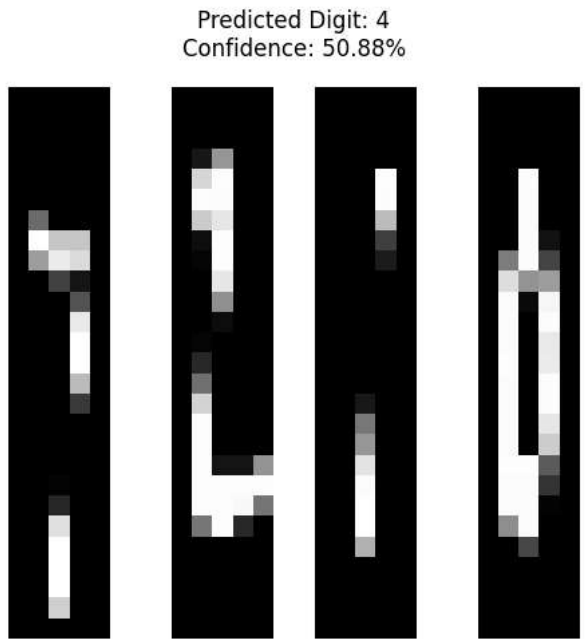
        plt.axis("off")

        plt.tight_layout()
        plt.show()

        print("\n================================")
        print("      CUSTOM IMAGE RESULT")
        print("================================")

        print(
            "Predicted Digit :",
            predicted_digit
        )

        print(
            "Confidence      :",
            f"{confidence * 100:.2f}%"
        )
```




```
================================
      CUSTOM IMAGE RESULT
================================
Predicted Digit : 4
Confidence      : 50.88%
```

Question Answers

Q1. What is image classification? How does it differ from object detection and image segmentation?

Answer:

Image classification assigns a single or multiple class labels to an image.

Classification: Identifies what is present in an image. Object Detection: Identifies objects and locates them using bounding boxes. Segmentation: Classifies individual pixels of an image.

Example:

For an image containing a dog:

Classification → Dog Detection → Dog + Bounding Box Segmentation → Pixels belonging to the dog

Q2. Explain the architecture and working principle of a CNN.

Answer:

A CNN consists mainly of:

Input Image ↓ Convolution Layer ↓ Activation Function ↓ Pooling Layer ↓ Convolution Layer ↓ Pooling Layer ↓ Flatten ↓ Fully Connected Layer ↓ Output Layer

The convolution layers extract features from the image. Pooling reduces the feature-map size. Fully connected layers perform classification, and the output layer provides the final class probabilities.

Q3. What is the role of Convolutional Layers, Pooling Layers, and Fully Connected Layers?

Answer:

Convolution Layer

Extracts important features such as edges, shapes and patterns.

Pooling Layer

Reduces the spatial dimensions of feature maps and decreases computation.

Fully Connected Layer

Combines extracted features and performs final classification.

Q4. Why is image normalization performed before training?

Answer:

Normalization converts pixel values into a smaller range, usually:

0–255 → 0–1

It helps:

Improve training stability. Speed up convergence. Make optimization easier. Provide consistent input values.

Q5. Explain ReLU and Softmax.

Answer:

ReLU

ReLU stands for Rectified Linear Unit.

Formula:

ReLU(x) = max(0,x)

It introduces non-linearity and helps the network learn complex patterns.

Softmax

Softmax converts output values into probabilities.

For example:

0 → 0.01 1 → 0.03 2 → 0.02 ... 7 → 0.91

The class with the highest probability becomes the predicted class.

Q6. What is a Confusion Matrix?

Answer:

A confusion matrix is a table used to compare:

Actual Class vs Predicted Class

For a digit classifier, it shows how many:

0s were correctly predicted as 0 1s were correctly predicted as 1 2s were correctly predicted as 2 etc.

It also helps identify misclassification between similar digits.

Q7. Differentiate Accuracy, Precision, Recall and F1-Score.

Answer:

Metric Meaning Accuracy Percentage of total predictions that are correct Precision How many predicted positive/class instances are actually correct Recall How many actual positive/class instances are correctly detected F1-Score Harmonic mean of Precision and Recall

Formulas:

Accuracy Accuracy = Correct Predictions / Total Predictions Precision Precision = TP / (TP + FP) Recall Recall = TP / (TP + FN) F1-Score F1 = 2 × (Precision × Recall) / (Precision + Recall)

Q8. Compare SIFT/HOG + SVM with CNN-based classification.

Answer:

Feature-based Method CNN Uses manually designed features Learns features automatically SIFT/HOG can be used Convolution layers extract features SVM commonly used for classification Neural network performs classification Feature engineering required Less manual feature engineering Usually suitable for smaller/simple tasks Effective for complex image tasks

The experiment sheet specifically asks for this comparison using examples such as SIFT/HOG + SVM.

Q9. Mention five real-world applications of CNN-based image classification.

Answer:

Handwritten Digit Recognition Face Recognition Biometric Authentication Medical Image Diagnosis Intelligent Document Processing

These application areas are also identified in the experiment sheet's purpose section.

Q10. How do Data Augmentation, Transfer Learning and Hyperparameter Tuning improve performance?

Answer:

Data Augmentation

Creates modified versions of training images using operations such as:

Rotation Scaling Flipping Cropping

It can increase training-data variety and help reduce overfitting.